

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import plotly.express as px
plt.style.use('seaborn-v0_8') # Better default style
plt.rcParams['figure.figsize'] = (10, 6) # Default figure size
```

✓ Pandas

- ✓ Pandas is a powerful, open-source data analysis and manipulation library built on top of NumPy. It provides:

DataFrame: 2D table-like structure (like Excel spreadsheet)

Series: 1D labeled array (single column of data)

Tools for reading/writing data from various formats

Data cleaning, transformation, and analysis capabilities

✓ Series

```
s1 = pd.Series(["a", "b", "c", "d"])
```

s1

```

  0
0  a
1  b
2  c
3  d
```

dtype: object

Start [coding](#) or [generate](#) with AI.

```
##Syntax: s2 = pd.Series([values],index=[indices])

s2 = pd.Series([10,20,30,40],index=["a", "b", "c", "d"])
s2
```

```

  0
a  10
b  20
c  30
d  40
```

dtype: int64

✓ From a Dict

```
#create a series using dictionary consisting name and age of the users

dictSeries = pd.Series({'Waleed':'20', 'Ahmed':17,'Ali':'25'})
print(dictSeries)
```

```

Waleed    20
Ahmed     17
Ali       25
dtype: object
```

✓ Retrieval/Selection of Series

```
## Single Label
```

```

print(s1[1])
print(dictSeries['Waleed'])
print(s1[3])

## List of Labels
print("\nList of Labels")
print(s1[[1,2]])
print(s2[["a","c"]])

##Filtering Conditions

# using mask
mask = s2>10
print(s2[mask])

##or
##directly conditioning in print
print(s2[s2>10])

```

```

b
20
d

List of Labels
1    b
2    c
dtype: object
a    10
c    30
dtype: int64
b    20
c    30
d    40
dtype: int64
b    20
c    30
d    40
dtype: int64

```

Series Attributes

```

print(s1.values)
print(s1.index)
print(s1.dtype)
print(s1.size)
print(s1.shape)
print(s1.ndim)

```

```

['a' 'b' 'c' 'd']
RangeIndex(start=0, stop=4, step=1)
object
4
(4,)
1

```

Series Operations

```

s = pd.Series([1,2,3,4,5])

print(s+10)
print(s*s)

# Statistical operations
print("Mean:", s.mean())
print("Sum:", s.sum())
print("Median:", s.median())
print("Describe:\n",s.describe())

```

```

0    11
1    12
2    13
3    14
4    15
dtype: int64
0     1
1     4
2     9
3    16
4    25
dtype: int64

```

```

Mean: 3.0
Sum: 15
Median: 3.0
Describe:
  count    5.000000
  mean     3.000000
  std      1.581139
  min      1.000000
  25%      2.000000
  50%      3.000000
  75%      4.000000
  max      5.000000
dtype: float64

```

▼ DataFrame

▼ From Dictionary

```

Dict = {
    'Name' : ['Waleed', 'Ali', 'Ahmed'],
    'Age' : [20, 30, 19],
    'Country' : ['PK', 'IN', 'US']
}

DF_Dict = pd.DataFrame(Dict)
print(DF_Dict)
print()
print(DF_Dict.values)
print()
print(DF_Dict.index)

```

	Name	Age	Country
0	Waleed	20	PK
1	Ali	30	IN
2	Ahmed	19	US

```

[['Waleed' 20 'PK']
 ['Ali' 30 'IN']
 ['Ahmed' 19 'US']]

```

```

RangeIndex(start=0, stop=3, step=1)

```

▼ From List of Lists

```

List = [
    ['Waleed', 20, 'Karachi', 800000],
    ['Ahmed', 40, 'Multan', 300000],
    ['Ali', 13, 'Lahore', 550000]
]

cols = ['Name', 'Age', 'City', 'Salary'] ##list for setting the custom columns name

DF_List = pd.DataFrame(List, columns=cols)
print(DF_List)
print()

##We can also give custom Index names

idx = ['emp01', 'emp02', 'emp03']
DF_List = pd.DataFrame(List, index=idx, columns=cols)

print('With Custom Index Names')
print(DF_List)

```

	Name	Age	City	Salary
0	Waleed	20	Karachi	800000
1	Ahmed	40	Multan	300000
2	Ali	13	Lahore	550000

```

With Custom Index Names
      Name  Age  City  Salary
emp01 Waleed  20 Karachi 800000
emp02 Ahmed  40 Multan 300000
emp03 Ali    13 Lahore 550000

```

▼ From CSV File

```
##ageGreater18 = DF_List["Age"]>18
# print(type(DF_List["Age"]))
```

```
mask = DF_List["Age"]>18
print(mask)
DF_List.loc[mask]
```

```
emp01    True
emp02    True
emp03    False
Name: Age, dtype: bool
```

	Name	Age	City	Salary
emp01	Waleed	20	Karachi	800000
emp02	Ahmed	40	Multan	300000

```
csvDF= pd.read_csv("elections.csv")
```

csvDF

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
182	2024	Donald Trump	Republican	77303568	win	49.808629
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772
184	2024	Jill Stein	Green	861155	loss	0.554864
185	2024	Robert Kennedy	Independent	756383	loss	0.487357
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895

187 rows x 6 columns

Next steps: [Generate code with csvDF](#) [New interactive sheet](#)

✓ Using Loc and iloc

```
#Use iloc to select the first 10 rows and only the 'Candidate' and 'Party' columns.
csvDF.iloc[:10,[2,3]]
```

```
#Use loc to select all rows from 2020 to 2024 (inclusive) and show only the 'Year', 'Candidate', and 'Result' columns.
colsData = csvDF.loc[:,["Year","Candidate","Result"]]
colsData[(colsData["Year"]>=2020) & (colsData["Year"]<=2024)]
```

```
#Use loc to find all Republican candidates who won elections
csvDF.loc[(csvDF["Party"]=="Republican") & (csvDF["Result"]=="win")]
csvDF.loc[(csvDF["Party"]=="Democratic") & (csvDF["%"] > 50.00) & (csvDF["Result"]=="loss")]
```

```
df= csvDF.copy()
df
```

```
df.loc[(df["Year"]<1900, 'Result')]="loss"
df["Result"]
```

```

      Result
0      loss
1      loss
2      loss
3      loss
4      loss
...      ...
182     win
183     loss
184     loss
185     loss
186     loss
187 rows × 1 columns

```

```
dtype: object
```

```

WinCandidates = (csvDF["Popular vote"]>30000) & (csvDF["Result"] == 'win')
# print(WinCandidates.loc['Year'])
# csvDF[WinCandidates,['Year']]
# print(csvDF[WinCandidates.loc["Year"]])

cols = csvDF.columns.values
print(cols)

print(csvDF.dtypes)

for i in cols:
    print(type(i))

```

```

['Year' 'Candidate' 'Party' 'Popular vote' 'Result' '%']
Year      int64
Candidate object
Party     object
Popular vote  int64
Result      object
%          float64
dtype: object
<class 'str'>
<class 'str'>
<class 'str'>
<class 'str'>
<class 'str'>
<class 'str'>

```

```

# csvDF.loc[WinCandidates.head]
# print(WinCandidates)

# mask = csvDF.loc[csvDF['Year']>2023]
# print(mask)

mask = csvDF[csvDF['Year']>2023]
print(mask)

```

	Year	Candidate	Party	Popular vote	Result	%
182	2024	Donald Trump	Republican	77303568	win	49.808629
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772
184	2024	Jill Stein	Green	861155	loss	0.554864
185	2024	Robert Kennedy	Independent	756383	loss	0.487357
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895

```

# csvDF[['Year','Candidate']]
mask = csvDF[csvDF['Year']>2000][['Year','Candidate']]
print(mask.head())

```

	Year	Candidate
156	2004	David Cobb
157	2004	George W. Bush
158	2004	John Kerry
159	2004	Michael Badnarik
160	2004	Michael Peroutka

```

DC = csvDF.loc[csvDF['Party']=="Democratic",'Candidate']
DC.head()

```

Candidate

- 2 Andrew Jackson
- 4 Andrew Jackson
- 8 Martin Van Buren
- 10 Martin Van Buren
- 13 James Polk

dtype: object

```
RW = csvDF.loc[(csvDF["Party"]=="Republican") & (csvDF["Result"] == "win") & (csvDF["Year"]>2000)]
RW
```

	Year	Candidate	Party	Popular vote	Result	%	
157	2004	George W. Bush	Republican	62040610	win	50.771824	
173	2016	Donald Trump	Republican	62984828	win	46.407862	
182	2024	Donald Trump	Republican	77303568	win	49.808629	

Next steps: [Generate code with RW](#) [New interactive sheet](#)

```
Lost50 = csvDF.loc[(csvDF["Result"]=="loss") & (csvDF["%"] > 50.00)]
Lost50
```

	Year	Candidate	Party	Popular vote	Result	%	
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122	
34	1876	Samuel J. Tilden	Democratic	4288546	loss	51.528376	
							

Next steps: [Generate code with Lost50](#) [New interactive sheet](#)

```
Margin2 = csvDF.loc[(csvDF["%"]>48) & (csvDF["%"]<52)]
Margin2
```

	Year	Candidate	Party	Popular vote	Result	%	
12	1844	Henry Clay	Whig	1300004	loss	49.250523	
13	1844	James Polk	Democratic	1339570	win	50.749477	
17	1852	Franklin Pierce	Democratic	1605943	win	51.013168	
33	1876	Rutherford Hayes	Republican	4034142	win	48.471624	
34	1876	Samuel J. Tilden	Democratic	4288546	loss	51.528376	
36	1880	James Garfield	Republican	4453337	win	48.369234	
37	1880	Winfield Scott Hancock	Democratic	4444976	loss	48.278422	
39	1884	Grover Cleveland	Democratic	4914482	win	48.884933	
40	1884	James G. Blaine	Republican	4856905	loss	48.312208	
45	1888	Grover Cleveland	Democratic	5534488	loss	48.656799	
53	1896	William McKinley	Republican	7112138	win	51.213817	
74	1916	Woodrow Wilson	Democratic	9126868	win	49.367987	
100	1948	Harry Truman	Democratic	24179347	win	49.601536	
111	1960	John Kennedy	Democratic	34220984	win	50.082561	
112	1960	Richard Nixon	Republican	34108157	loss	49.917439	
122	1976	Gerald Ford	Republican	39148634	loss	48.199499	
123	1976	Jimmy Carter	Democratic	40831881	win	50.271900	
131	1980	Ronald Reagan	Republican	43903230	win	50.897944	
144	1996	Bill Clinton	Democratic	47400125	win	49.296938	
151	2000	Al Gore	Democratic	50999897	loss	48.491813	
157	2004	George W. Bush	Republican	62040610	win	50.771824	
158	2004	John Kerry	Democratic	59028444	loss	48.306775	
168	2012	Barack Obama	Democratic	65915795	win	51.258484	
176	2016	Hillary Clinton	Democratic	65853514	loss	48.521539	
178	2020	Joseph Biden	Democratic	81268924	win	51.311515	
182	2024	Donald Trump	Republican	77303568	win	49.808629	
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772	

Next steps:

[Generate code with Margin2](#)

[New interactive sheet](#)

```
ThirdParty = csvDF.loc[~((csvDF["Party"]=="Democratic") | (csvDF["Party"]=="Republican"))]
ThirdParty
```

	Year	Candidate	Party	Popular vote	Result	%	
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122	
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878	
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073	
5	1832	Henry Clay	National Republican	484205	loss	37.603628	
6	1832	William Wirt	Anti-Masonic	100715	loss	7.821583	
...	...	...	...	...	...	...	
180	2020	Jo Jorgensen	Libertarian	1865724	loss	1.177979	
181	2020	Howard Hawkins	Green	405035	loss	0.255731	
184	2024	Jill Stein	Green	861155	loss	0.554864	
185	2024	Robert Kennedy	Independent	756383	loss	0.487357	
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895	

97 rows × 6 columns

Next steps:

[Generate code with ThirdParty](#)

[New interactive sheet](#)

▽ Group By

- ✓ Group By is one of the most powerful operations in pandas. It allows you to:

Split data into groups based on criteria

Apply functions to each group independently

Combine the results back together

```
elect = csvDF.copy()
# groupedParty = elect.groupby("Party")
# groupedParty.size()

## Basic Aggregation

#### Average vote percentage by party
party_group = elect.groupby("Party")["%"].mean()
## or party_group = elect.groupby("Party")["%"].agg(['mean'])
party_group

## Multiple Aggregation

#### Multiple Aggregation for each party
partyDetails = elect.groupby("Party")["%"].agg(['mean', 'max', 'min', 'count'])
partyDetails

##Grouping by Multiple Columns

#### Group by both Party and Result
MultiGroup = elect.groupby(["Party", "Result"])["%"].mean()
MultiGroup

win_loss_count = elect.groupby(['Party', 'Result']).size()
print(win_loss_count.head(5))
```

Party	Result
American	loss 2
American Independent	loss 3
Anti-Masonic	loss 1
Anti-Monopoly	loss 1
Citizens	loss 1

dtype: int64

```
partyAvg = elect.groupby('Party')['%'].mean().count()
partyAvg
```

np.int64(37)

```
#Count how many times each party appears in the dataset
partyTimes = elect.groupby('Party').size()
partyTimes

#Calculate the total popular votes received by each party across all years (sum)
totalPopular = elect.groupby('Party')['Popular vote'].sum()
totalPopular.head()

#For each election year, find how many candidates participated (count or size)
candCounts = elect.groupby("Year")["Candidate"].count()
candCounts.head()

#Show the maximum vote percentage achieved by each party
maxVotes = elect.groupby("Party")["%"].max()
maxVotes.head()
```

	%
Party	
American	21.554001
American Independent	13.571218
Anti-Masonic	7.821583
Anti-Monopoly	1.335838
Citizens	0.270182

dtype: float64

- ✓ Filtering by Group By

```
data = csvDF.head(4)
dt = data.copy()
print(dt)

dt.loc[len(dt)] = [2022, 'Waleed Afzal', 'National', 99999, 'win', 87.3]
print(dt)
```

```
Year      Candidate      Party  Popular vote  Result  \
0  1824    Andrew Jackson  Democratic-Republican    151271    loss
1  1824  John Quincy Adams  Democratic-Republican    113142    win
2  1828    Andrew Jackson      Democratic    642806    win
3  1828  John Quincy Adams  National Republican    500897    loss

%
0  57.210122
1  42.789878
2  56.203927
3  43.796073
Year      Candidate      Party  Popular vote  Result  \
0  1824    Andrew Jackson  Democratic-Republican    151271    loss
1  1824  John Quincy Adams  Democratic-Republican    113142    win
2  1828    Andrew Jackson      Democratic    642806    win
3  1828  John Quincy Adams  National Republican    500897    loss
4  2022    Waleed Afzal      National      99999    win

%
0  57.210122
1  42.789878
2  56.203927
3  43.796073
4  87.300000
```

```
##Keep parties whose candidates average more than 20% of the vote
high_avg_parties = elect.groupby("Party").filter(lambda group: group["%"].mean()>20)
print(high_avg_parties["Party"].unique())

print()
## Keep parties that have achieved at least 40% in any election
max40 = elect.groupby("Party").filter(lambda group: group["%"].max()>40)
print(max40["Party"].unique())

print()
## Keep parties that have candidates in elections from 1970 onwards
OldParties = elect.groupby("Party").filter(lambda group: group["Year"].max()>1970)
print(OldParties)
```

```
['Democratic-Republican' 'Democratic' 'National Republican' 'Whig'
'Republican' 'Northern Democratic' 'National Union' 'Liberal Republican']

['Democratic-Republican' 'Democratic' 'National Republican' 'Whig'
'Republican' 'National Union' 'Liberal Republican']
```

```
Year      Candidate      Party  Popular vote  Result  %
2  1828    Andrew Jackson      Democratic    642806    win  56.203927
4  1832    Andrew Jackson      Democratic    702735    win  54.574789
8  1836  Martin Van Buren      Democratic    763291    win  52.272472
10 1840  Martin Van Buren      Democratic    1128854    loss  46.948787
13 1844    James Polk      Democratic    1339570    win  50.749477
..   ..      ...      ...      ...      ...
182 2024    Donald Trump      Republican    77303568    win  49.808629
183 2024    Kamala Harris      Democratic    75019230    loss  48.336772
184 2024    Jill Stein      Green      861155    loss  0.554864
185 2024    Robert Kennedy      Independent    756383    loss  0.487357
186 2024    Chase Oliver  Libertarian Party    650130    loss  0.418895
```

[135 rows x 6 columns]

GROUP BY QUESTIONS

```
##Find the average vote percentage for each political party.
elect.groupby("Party")["%"].mean()

##Calculate the win rate (percentage of wins) for each major party (Democratic and Republican)
elect['Win'] = elect["Result"]=="win"
Parties = elect.groupby('Party')['Win'].mean()
Parties.loc[["Democratic", "Republican"]]

## Group by year and find which year had the most candidates.
mostCands = elect.groupby(["Year"])["Candidate"].count()
year_with_most = mostCands.idxmax() #### Find the year with most candidates
most_cands_count = mostCands.max() ####calculate the max of the count of the candidates
print("Year: ", year_with_most, " | Candidates: ", most_cands_count)

##For each party, calculate the minimum, maximum, and average popular vote received.
partyDetails = elect.groupby("Party")["Popular vote"].agg(["min", "max", "mean"])
```

```
print(partyDetails.head(10))

## Find the candidate with the highest vote percentage in each party.
print()
highestVotes = elect.groupby("Candidate")["%"].max()
print(highestVotes.idxmax())
print(highestVotes.max())
```

Year: 1996 || Candidates: 7

	min	max	mean
Party			
American	158271	873053	5.156620e+05
American Independent	170274	9901118	3.724087e+06
Anti-Masonic	100715	100715	1.007150e+05
Anti-Monopoly	134294	134294	1.342940e+05
Citizens	233052	233052	2.330520e+05
Communist	103307	103307	1.033070e+05
Constitution	143630	203091	1.821570e+05
Constitutional Union	590901	590901	5.909010e+05
Democratic	642806	81268924	2.386054e+07
Democratic-Republican	113142	151271	1.322065e+05

Lyndon Johnson
61.34470329

elect

	Year	Candidate	Party	Popular vote	Result	%	Win	
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122	False	
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878	True	
2	1828	Andrew Jackson	Democratic	642806	win	56.203927	True	
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073	False	
4	1832	Andrew Jackson	Democratic	702735	win	54.574789	True	
...	...	...	...	...	...	...	...	
182	2024	Donald Trump	Republican	77303568	win	49.808629	True	
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772	False	
184	2024	Jill Stein	Green	861155	loss	0.554864	False	
185	2024	Robert Kennedy	Independent	756383	loss	0.487357	False	
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895	False	

187 rows x 7 columns

Next steps: [Generate code with elect](#) [New interactive sheet](#)

- ✓ Pivot Table
- ✓ `pd.pivot_table(data, values=None, index=None, columns=None, aggfunc='mean', fill_value=None)`

Parameter Description Example

- `data` DataFrame to use elect
- `values` Column(s) to aggregate '%', 'Popular vote'
- `index` Column(s) for row labels 'Year', 'Party'
- `columns` Column(s) for column labels 'Result', 'Party'
- `aggfunc` Aggregation function 'mean', 'sum', 'count'
- `fill_value` Value to replace NaN 0, -1

```
# Basic pivot: Average vote % by Party and Result

p1 = elect.pivot_table(
    index = "Year",
    columns = "Result",
    values = "%",
    aggfunc = "mean"
)

p1.head(10)
```

Result	loss	win
Year		

1824	57.210122	42.789878
1828	43.796073	56.203927
1832	22.712605	54.574789
1836	23.863764	52.272472
1840	46.948787	53.051213
1844	49.250523	50.749477
1848	26.345352	47.309296
1852	24.493416	51.013168
1856	27.346960	45.306080
1860	20.100197	39.699408

Next steps: [Generate code with p1](#) [New interactive sheet](#)

```
# Count candidates by Party and Result

pivot2 = elect.pivot_table(
    index="Party",
    columns = "Result",
    values = "Candidate",
    aggfunc = "count",
    fill_value = "0"
)

pivot2.head()
```

	Result	loss	win
Party			
American		2	0
American Independent		3	0
Anti-Masonic		1	0
Anti-Monopoly		1	0
Citizens		1	0

Next steps: [Generate code with pivot2](#) [New interactive sheet](#)

```
# Multiple statistics for Popular vote by Party

p3 = elect.pivot_table(
    index = "Party",
    aggfunc = ['mean', 'max', 'min'],
    values = "Popular vote",
    fill_value = "0"
)

p3.head()
```

	mean	max	min
Party	Popular vote	Popular vote	Popular vote
American	5.156620e+05	873053	158271
American Independent	3.724087e+06	9901118	170274
Anti-Masonic	1.007150e+05	100715	100715
Anti-Monopoly	1.342940e+05	134294	134294
Citizens	2.330520e+05	233052	233052

Next steps: [Generate code with p3](#) [New interactive sheet](#)

```
# Students table
students = pd.DataFrame({
    'student_id': [101, 102, 103, 104],
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'age': [16, 17, 16, 18]
})

# Scores table
scores = pd.DataFrame({
    'student_id': [101, 102, 103, 105],
    'math_score': [85, 92, 78, 88],
    'english_score': [90, 85, 92, 79]
})

print(students)
print(scores)
```

```

  student_id  name  age
0         101  Alice   16
1         102   Bob   17
2         103 Charlie   16
3         104  David   18
  student_id  math_score  english_score
0         101          85             90
1         102          92             85
2         103          78             92
3         105          88             79
```

```
merged = pd.merge(left=students, right=scores)
merged

# or

merged = pd.merge(left=students, right=scores, left_on="student_id", right_on="student_id")
merged
```

	student_id	name	age	math_score	english_score
0	101	Alice	16	85	90
1	102	Bob	17	92	85
2	103	Charlie	16	78	92

Next steps: [Generate code with merged](#) [New interactive sheet](#)

pwd

'/content'

This is formatted as code

Practice

```
emp = pd.read_csv("Employee.csv")
emp
```

	Employee_ID	Name	Age	Department	Salary	Joining_Date	Gender	Experience_Years	Performance_Score	Work_Location
0	1	Emp_1	56.0	Finance	40575.0	1/31/2015	Female	19	3.0	Tokyo
1	2	Emp_2	46.0	IT	58423.0	2/28/2015	M	24	1.0	Remote
2	3	Emp_3	32.0	IT	NaN	3/31/2015	Female	25	5.0	New York
3	4	Emp_4	60.0	Finance	40497.0	4/30/2015	Female	10	4.0	Remote
4	5	Emp_5	25.0	HR	83146.0	5/31/2015	M	5	4.0	Remote
...	...	...	...	...	...	...	...	...	...	...
195	196	Emp_196	49.0	Operations	50694.0	4/30/2031	Male	6	2.0	Remote
196	197	Emp_197	0.0	Marketing	48953.0	5/31/2031	Female	8	2.0	London
197	198	Emp_198	41.0	IT	68692.0	6/30/2031	F	21	1.0	New York
198	199	Emp_199	58.0	HR	51790.0	7/31/2031	Male	21	4.0	New York
199	200	Emp_200	29.0	IT	NaN	8/31/2031	Female	10	1.0	New York

200 rows × 10 columns

Next steps: [Generate code with emp](#) [New interactive sheet](#)

```
## Select all employees who work in the "IT" department.
IT_emp = emp.loc[emp["Department"]=="IT"]
print(IT_emp.head())

## 2. Find employees with Salary greater than 80,000 using `.loc[]`

print("\n2. =====>")
Sal80 = emp.loc[emp["Salary"]>80000]
print(Sal80.head())

# 3. Select only Name and Department columns for employees aged 30-40 using `.loc[]`
print("\n3. =====>")
midAged = emp.loc[(emp["Age"]>=30) & (emp["Age"]<=40)]
print(midAged.loc[:,["Name","Department"]].head())

# 4. Get all female employees from Berlin using `.loc[]`
print("\n4. =====>")
femalesBerlin = emp.loc[(emp["Gender"] == "Female") | (emp["Gender"] == "F") & emp["Work_Location"] == "Berlin"]
print(femalesBerlin.head())

# 5. Select employees with Performance_Score of 5 using `.loc[]`
print("\n5. =====>")
bestScored = emp.loc[emp["Performance_Score"] == 5]
print(bestScored.head())
```

	Employee_ID	Name	Age	Department	Salary	Joining_Date	Gender	\
1	2	Emp_2	46.0	IT	58423.0	2/28/2015	M	
2	3	Emp_3	32.0	IT	NaN	3/31/2015	Female	
6	7	Emp_7	56.0	IT	NaN	7/31/2015	Male	
12	13	Emp_13	53.0	IT	56963.0	1/31/2016	Female	
19	20	Emp_20	NaN	IT	91669.0	8/31/2016	Male	

	Experience_Years	Performance_Score	Work_Location
1	24	1.0	Remote
2	25	5.0	New York
6	6	3.0	Remote
12	10	4.0	Remote
19	24	3.0	London

```
2. =====>
Employee_ID  Name  Age  Department  Salary  Joining_Date  Gender  \
4            5  Emp_5  25.0         HR    83146.0    5/31/2015      M
9           10 Emp_10  28.0  Operations    95219.0   10/31/2015  Female
19          20 Emp_20   NaN         IT    91669.0    8/31/2016   Male
31          32 Emp_32  45.0         IT    81788.0    8/31/2017  Female
36          37 Emp_37  20.0         IT   682760.0    1/31/2018  Female
```

	Experience_Years	Performance_Score	Work_Location
4	5	4.0	Remote
9	1	1.0	Tokyo
19	24	3.0	London
31	27	1.0	New York
36	16	3.0	London

```
3. =====>
Name Department
2  Emp_3       IT
5  Emp_6  Marketing
7  Emp_8       HR
```

```
8 Emp_9 Finance
16 Emp_17 Marketing
```

```
4. =====>
```

```
Empty DataFrame
```

```
Columns: [Employee_ID, Name, Age, Department, Salary, Joining_Date, Gender, Experience_Years, Performance_Score, Work_Location]
```

```
Index: []
```

```
5. =====>
```

	Employee_ID	Name	Age	Department	Salary	Joining_Date	Gender	\
2	3	Emp_3	32.0	IT	NaN	3/31/2015	Female	
34	35	Emp_35	64.0	Finance	50554.0	11/30/2017	Female	
37	38	Emp_38	54.0	Operations	49348.0	2/28/2018	Female	
77	78	Emp_78	31.0	Marketing	80401.0	6/30/2021	Female	
103	104	Emp_104	34.0	HR	74644.0	8/31/2023	Male	

	Experience_Years	Performance_Score	Work_Location
2	25	5.0	New York
34	16	5.0	Tokyo
37	19	5.0	New York
77	29	5.0	London
103	22	5.0	Berlin

```
##GROUP BY
```

```
print("\nFind the average Salary by Department\n")
avgSal = emp.groupby("Department")["Salary"].mean()
print(avgSal)
```

```
print("\nCount the number of employees in each Work_Location\n")
count_Emps = emp.groupby("Work_Location")["Employee_ID"].count()
print(count_Emps)
```

```
print("\nCalculate the maximum Age in each Department\n")
MaxAge = emp.groupby("Department")["Age"].max()
print(MaxAge)
```

```
print("\nCalculate the standard deviation of Salary by Work_Location")
salarySTD = emp.groupby("Work_Location")["Salary"].std()
print(salarySTD)
```

```
Find the average Salary by Department
```

```
Department
Finance      68856.763158
HR           64036.916667
IT           73722.541667
Marketing     82008.166667
Operations    85795.576923
Name: Salary, dtype: float64
```

```
Count the number of employees in each Work_Location
```

```
Work_Location
Berlin      32
London      38
New York    55
Remote      37
Tokyo       38
Name: Employee_ID, dtype: int64
```

```
Calculate the maximum Age in each Department
```

```
Department
Finance      64.0
HR           61.0
IT           62.0
Marketing     64.0
Operations    64.0
Name: Age, dtype: float64
```

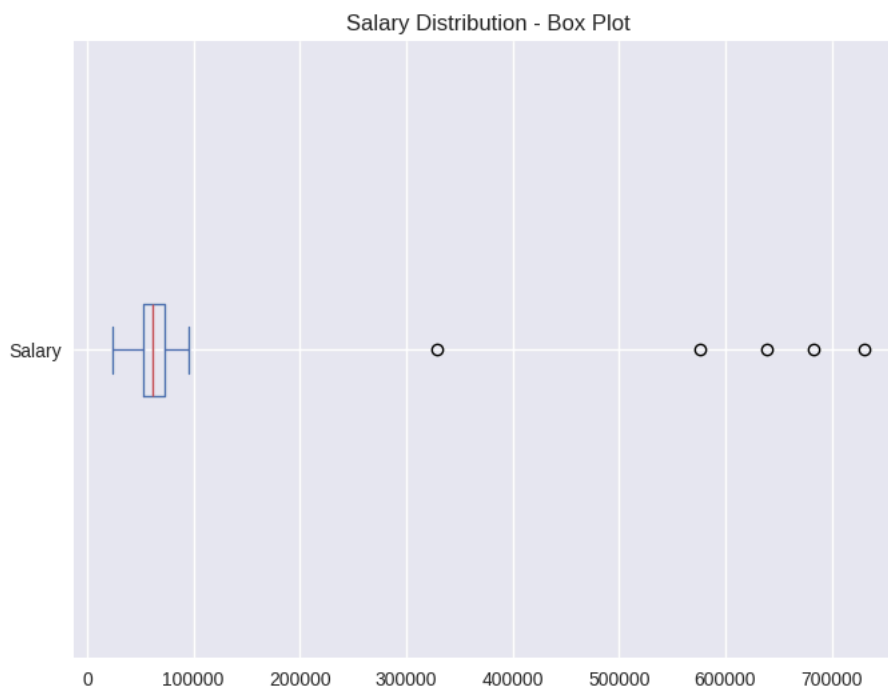
```
Calculate the standard deviation of Salary by Work_Location
```

```
Work_Location
Berlin      119893.911586
London      101262.881437
New York    89967.465063
Remote      13569.582898
Tokyo       87291.635063
Name: Salary, dtype: float64
```

```
Age_Dept = emp.groupby("Department")["Age"].max()
px.line(Age_Dept, y="Age")
```

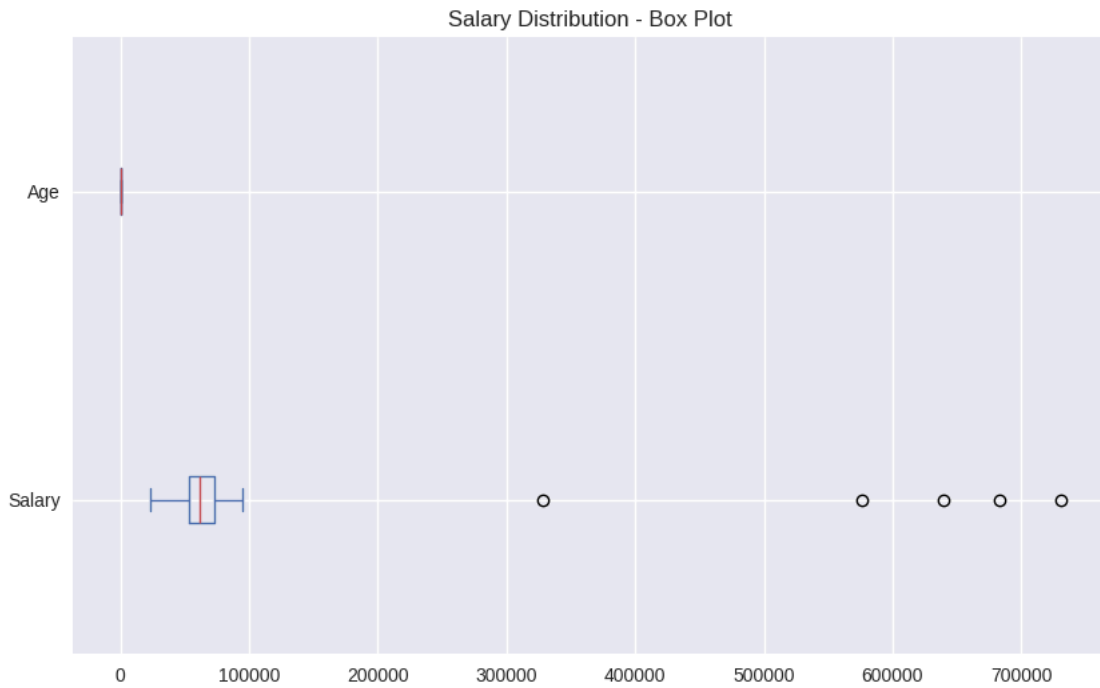


```
plt.figure(figsize=(8, 6))
emp['Salary'].plot(kind='box', vert=False)
plt.title('Salary Distribution - Box Plot')
plt.show()
```

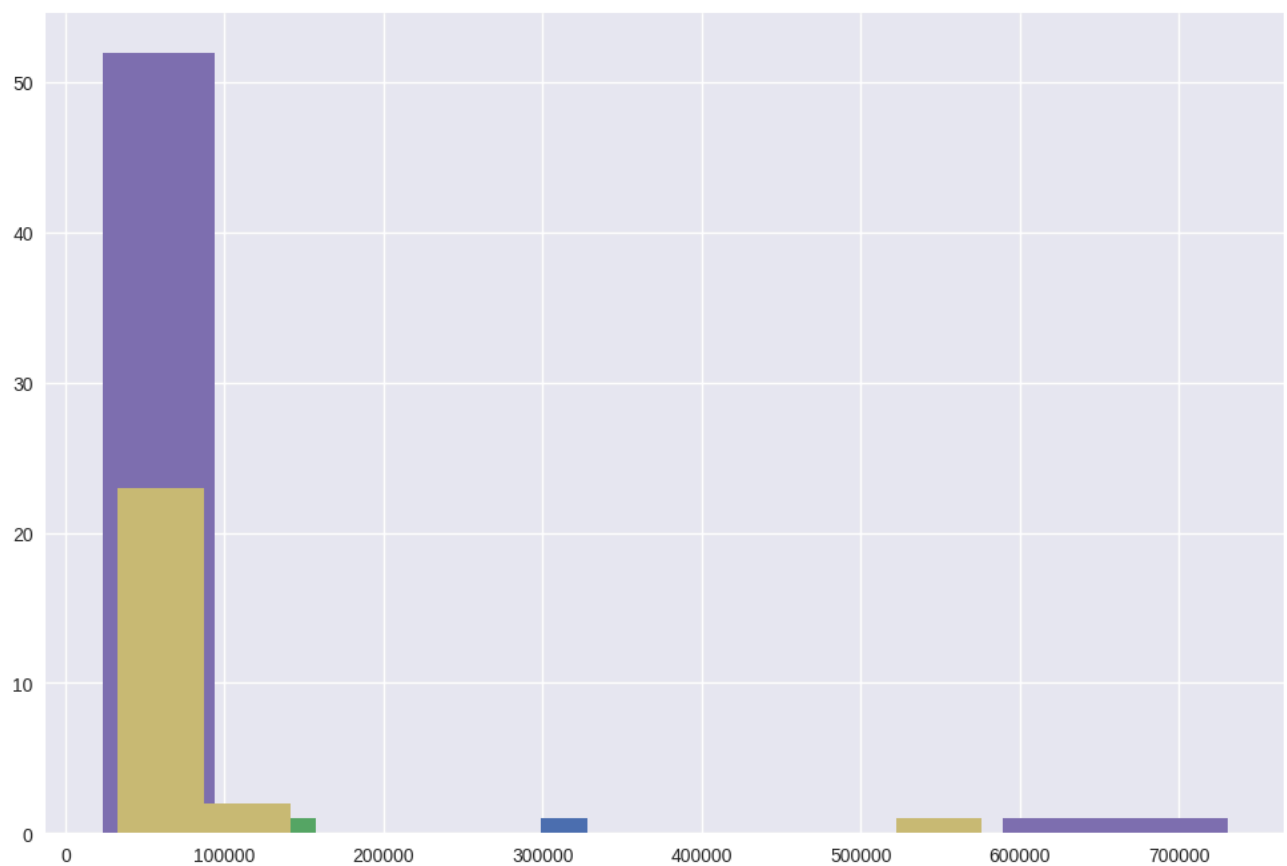


```
plt.figure(figsize=(8, 6))
emp[['Salary', 'Age']].plot(kind='box', vert=False)
plt.title('Salary Distribution - Box Plot')
plt.show()
```

<Figure size 800x600 with 0 Axes>



```
plt.figure(figsize=(12, 8))
for department in emp['Department'].unique():
    department_data = emp[emp['Department'] == department]['Salary']
    plt.hist(department_data, label=department)
```



##Question 1:

```
emp
print(emp.shape)
print(emp.size)
print(emp.ndim)
print(emp.columns)

print("\nCount of Missing Values Per Column")
countNa = emp.isnull().sum()
print(countNa)
```

```

print("\nCount of duplicates:")
countDupli = emp.duplicated().sum()
print(countDupli)

print("\nSelect only the Name, Age, and Salary columns from the dataset.")
emp.loc[:, ["Name", "Age", "Salary"]]

print("Extract the first 10 rows and the last 5 rows of the dataset.")
print("\nFirst 10 Rows: ", emp.head(10))
print("\nLast 5 Rows: ", emp.tail(5))

print("\nSlice rows where Age is between 30 and 40 and Department is 'Engineering'.")
age30_40 = emp.loc[(emp["Age"]>=30) | (emp["Age"]<=40) & (emp["Department"] == "Engineering")]
print(age30_40)

print("\nSelect all employees from 'New York' with Performance_Score above 8.")
NY = emp.loc[(emp["Work_Location"]=="New York") & (emp["Performance_Score"]>8)]
print(NY)

print("\nSelect the first 5 rows and the last 5 rows of numerical columns only.")
numerical = list()
for i in emp.columns:
    if((emp[i].dtype == 'int64') or (emp[i].dtype == 'float64')):
        numerical.append(i)

print("First 5 Rows: \n",emp.loc[:,numerical].head(5))
print("Last 5 Rows:\n ",emp.loc[:,numerical].tail(5))

print("\nIdentify and fix:\n\t• Mixed categorical values (e.g., 'Male', 'M'→ 'Male')\n")
emp.loc[:, "Gender"] = emp["Gender"].replace({"M": "Male", "F": "Female"})
emp["Gender"].fillna("Male")
print(emp["Gender"])

print("\n\t• Impossible values (e.g., negative ages, unrealistic salaries).\n")
print("Max: ", emp["Age"].max())
print("Min: ", emp["Age"].min())

for i in emp.index:
    if(emp.loc[i, "Age"]<=0):
        emp.loc[i, "Age"]=18

print(emp["Age"])

print("\n\t\tFixing Salaries\n")
print("Max: ", emp["Salary"].max())
print("Min: ", emp["Salary"].min())

for i in emp.index:
    if(emp.loc[i, "Salary"]>100000 or emp.loc[i, "Salary"] <=0):
        emp.loc[i, "Salary"] = 20000

emp["Salary"] = emp["Salary"].fillna(20000)

print(emp["Salary"])

print("\n\t• Standardize Capitalization\n")
emp.loc[:, "Department"] = emp["Department"].str.upper()
emp["Department"]

```



```
(200, 10)
2000
2
Index(['Employee_ID', 'Name', 'Age', 'Department', 'Salary', 'Joining_Date',
      'Gender', 'Experience_Years', 'Performance_Score', 'Work_Location'],
      dtype='object')
```

Count of Missing Values Per Column

Employee_ID	0
Name	0
Age	8
Department	0
Salary	10
Joining_Date	0
Gender	18
Experience_Years	0
Performance_Score	14
Work_Location	0

dtype: int64

Count of duplicates:
0

Select only the Name, Age, and Salary columns from the dataset.
Extract the first 10 rows and the last 5 rows of the dataset.

First 10 Rows:

	Employee_ID	Name	Age	Department	Salary	Joining_Date	Gender	\
0	1	Emp_1	56.0	Finance	40575.0	1/31/2015	Female	
1	2	Emp_2	46.0	IT	58423.0	2/28/2015	M	
2	3	Emp_3	32.0	IT	NaN	3/31/2015	Female	
3	4	Emp_4	60.0	Finance	40497.0	4/30/2015	Female	
4	5	Emp_5	25.0	HR	83146.0	5/31/2015	M	
5	6	Emp_6	38.0	Marketing	79848.0	6/30/2015	Male	
6	7	Emp_7	56.0	IT	NaN	7/31/2015	Male	
7	8	Emp_8	36.0	HR	53015.0	8/31/2015	M	
8	9	Emp_9	40.0	Finance	60877.0	9/30/2015	F	
9	10	Emp_10	28.0	Operations	95219.0	10/31/2015	Female	

	Experience_Years	Performance_Score	Work_Location
0	19	3.0	Tokyo
1	24	1.0	Remote
2	25	5.0	New York
3	10	4.0	Remote
4	5	4.0	Remote
5	1	4.0	New York
6	6	3.0	Remote
7	17	4.0	Tokyo
8	26	3.0	Remote
9	1	1.0	Tokyo

Last 5 Rows:

	Employee_ID	Name	Age	Department	Salary	Joining_Date	Gender	\
195	196	Emp_196	49.0	Operations	50694.0	4/30/2031	Male	
196	197	Emp_197	0.0	Marketing	48953.0	5/31/2031	Female	
197	198	Emp_198	41.0	IT	68692.0	6/30/2031	F	
198	199	Emp_199	58.0	HR	51790.0	7/31/2031	Male	
199	200	Emp_200	29.0	IT	NaN	8/31/2031	Female	

	Experience_Years	Performance_Score	Work_Location
195	6	2.0	Remote
196	8	2.0	London
197	21	1.0	New York
198	21	4.0	New York
199	10	1.0	New York

Slice rows where Age is between 30 and 40 and Department is "Engineering".

	Employee_ID	Name	Age	Department	Salary	Joining_Date	Gender	\
0	1	Emp_1	56.0	Finance	40575.0	1/31/2015	Female	
1	2	Emp_2	46.0	IT	58423.0	2/28/2015	M	
2	3	Emp_3	32.0	IT	NaN	3/31/2015	Female	
3	4	Emp_4	60.0	Finance	40497.0	4/30/2015	Female	
5	6	Emp_6	38.0	Marketing	79848.0	6/30/2015	Male	
..	...	...	...	...	...	...	...	
193	194	Emp_194	41.0	HR	78069.0	2/28/2031	Female	
194	195	Emp_195	32.0	HR	72770.0	3/31/2031	Female	
195	196	Emp_196	49.0	Operations	50694.0	4/30/2031	Male	
197	198	Emp_198	41.0	IT	68692.0	6/30/2031	F	
198	199	Emp_199	58.0	HR	51790.0	7/31/2031	Male	

	Experience_Years	Performance_Score	Work_Location
0	19	3.0	Tokyo
1	24	1.0	Remote
2	25	5.0	New York
3	10	4.0	Remote
4	5	4.0	Remote
5	1	4.0	New York
6	6	3.0	Remote
7	17	4.0	Tokyo
8	26	3.0	Remote
9	1	1.0	Tokyo

Question 2:
print(emp)

```
print("\nHandling Missing Values:\n")
ageMean = emp["Age"].mean()
emp["Age"] = emp["Age"].fillna(ageMean)
print(emp["Age"])
```